Erste Schritte mit Ti*k*Z: Grundlagen des Grafikpakets

Martin Quandt

13. Mai 2025

1 Einführung in Ti*k*Z

Ti*k*Z ist ein leistungsstarkes Grafikpaket für LaTeX, dessen Name für „Ti*k*Z ist kein Zeichenprogramm“ steht. Es ermöglicht die Erstellung professioneller Grafiken direkt im LaTeX-Dokument. Dieses Nachschlagewerk erläutert die Grundlagen von Ti*k*Z und gibt praktische Beispiele für den Einstieg.

2 Einbinden von Ti*k*Z

Ti*k*Z wird in der Präambel eingebunden. Häufig werden zusätzlich Bibliotheken geladen, die spezielle Funktionen bereitstellen.

```
\usepackage{tikz}                   % Ti\textit{k}Z-Paket laden
\usepackage{pgfplots}               % Erweiterte Plotfunktionen (optional)
\pgfplotsset{compat=1.17}           % Kompatibilitaetseinstellung

% Nuetzliche Ti\textit{k}Z-Bibliotheken
\usetikzlibrary{arrows.meta}        % Erweiterte Pfeilspitzen
\usetikzlibrary{calc}               % Rechnen mit Koordinaten
\usetikzlibrary{angles}             % Winkeldarstellung
\usetikzlibrary{quotes}             % Vereinfachte Beschriftungen
\usetikzlibrary{shapes}             % Verschiedene Formen fuer Knoten
\usetikzlibrary{patterns}           % Verschiedene Fuellmuster
\usetikzlibrary{positioning}        % Verbesserte Positionierung von Knoten
\usetikzlibrary{intersections}      % Schnittpunkte finden
```

3 Grundstruktur einer Ti*k*Z-Grafik

Jede Ti*k*Z-Grafik wird in einer `tikzpicture`-Umgebung erstellt. Der Ursprung des Koordinatensystems liegt standardmäßig bei $(0, 0)$.

```
\begin{tikzpicture}
    % Hier kommen die Zeichenbefehle
\end{tikzpicture}
```

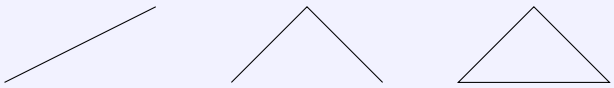
4 Grundlegende Zeichenbefehle

4.1 Linien und Pfade zeichnen

Der wichtigste Befehl in Ti*k*Z ist `\draw`, der zum Zeichnen von Linien und Pfaden verwendet wird.

```
\begin{tikzpicture}
    % Einfache Linie von (0,0) nach (2,1)
    \draw (0,0) -- (2,1);
    % Mehrere verbundene Linien (Pfad)
    \draw (3,0) -- (4,1) -- (5,0);
    % Geschlossener Pfad (Dreieck) mit cycle
    \draw (6,0) -- (7,1) -- (8,0) -- cycle;
\end{tikzpicture}
```

Ausgabe



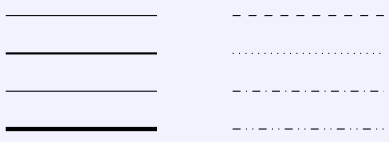
4.2 Linientypen und -stärke

Ti*k*Z bietet verschiedene Möglichkeiten, das Aussehen von Linien zu verändern.

```
\begin{tikzpicture}
    % Verschiedene Linienstile
    \draw (0,0) -- (2,0);                          % Standard-Linie
    \draw[thick] (0,-0.5) -- (2,-0.5);             % Dicke Linie
    \draw[thin] (0,-1) -- (2,-1);                  % Duenne Linie
    \draw[ultra thick] (0,-1.5) -- (2,-1.5);       % Sehr dicke Linie

    % Verschiedene Linienmuster
    \draw[dashed] (3,0) -- (5,0);                  % Gestrichelte Linie
    \draw[dotted] (3,-0.5) -- (5,-0.5);            % Gepunktete Linie
    \draw[dash dot] (3,-1) -- (5,-1);              % Strich-Punkt-Linie
    \draw[dash dot dot] (3,-1.5) -- (5,-1.5);      % Strich-Punkt-Punkt-Linie
\end{tikzpicture}
```

Ausgabe



4.3 Flächen zeichnen und füllen

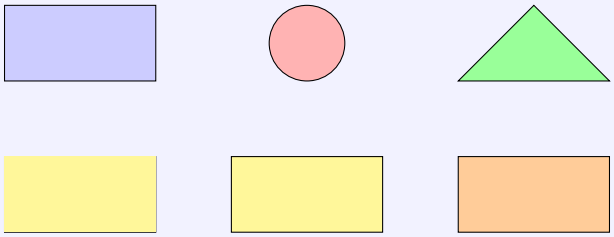
Mit Ti*k*Z können Flächen gefüllt werden. Wichtig ist, dass die Füllung **vor** dem Zeichnen der Umrandung definiert wird, damit die Linien nicht mitgefärbt werden.

```
\begin{tikzpicture}
     % Rechteck mit blauer Fuellung und schwarzer Umrandung
    \draw[fill=blue!20] (0,0) rectangle (2,1);
```

```
    % Kreis mit roter Fuellung
    \draw[fill=red!30] (4,0.5) circle (0.5);
    % Dreieck mit gruener Fuellung
    \draw[fill=green!40] (6,0) -- (7,1) -- (8,0) -- cycle;

    % Falsche Reihenfolge (NICHT EMPFOHLEN):
    % Die Linie wird mitgefaerbt
    \draw (0,-2) rectangle (2,-1);
    \fill[yellow!50] (0,-2) rectangle (2,-1);
    % Richtige Reihenfolge:
    \fill[yellow!50] (3,-2) rectangle (5,-1);
    \draw (3,-2) rectangle (5,-1);
    % Alternative mit einem Befehl:
    \filldraw[fill=orange!40, draw=black] (6,-2) rectangle (8,-1);
\end{tikzpicture}
```

Ausgabe



4.4 Grundformen

TikZ bietet verschiedene Grundformen, die einfach gezeichnet werden können.

```
\begin{tikzpicture}
    % Rechteck: (x1,y1) und (x2,y2) sind gegenueberliegende Ecken
    \draw (0,0) rectangle (2,1);
    % Kreis: (x,y) ist der Mittelpunkt, der Radius wird in Klammern angegeben
    \draw (3,0.5) circle (0.5);
    % Ellipse: (x,y) ist der Mittelpunkt, die Halbachsen werden in Klammern angegeben
    \draw (5,0.5) ellipse (0.7 and 0.4);
    % Bogen: Start- und Endwinkel werden in Grad angegeben
    \draw (7,0.5) arc (0:270:0.5);
\end{tikzpicture}
```

Ausgabe



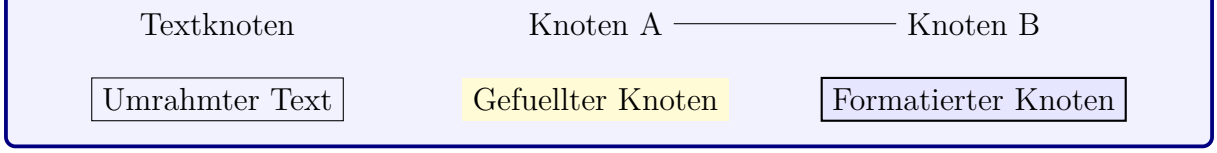
5 Die `node`-Funktion

Besonders mächtig in TikZ ist die `node`-Funktion, die es ermöglicht, Elemente (Text, Symbole, etc.) an bestimmten Positionen zu platzieren. Knoten sind die Grundlage für komplexere Strukturen wie Diagramme oder Graphen.

5.1 Grundlegende Verwendung von Knoten

```
\begin{tikzpicture}
    %Einfacher Textknoten an Position (0,0)
    \node at (0,0) {Textknoten};
    %Knoten mit Namen 'A' zur spaeteren Referenzierung
    \node (A) at (5,0) {Knoten A};
    %Knoten mit Namen 'B'
    \node (B) at (10,0) {Knoten B};
    %Verbindung zwischen benannten Knoten
    \draw (A) -- (B);
    %Knoten mit Rahmen
    \node[draw] at (0,-1) {Umrahmter Text};
    %Knoten mit gefuelltem Hintergrund
    \node[fill=yellow!20] at (5,-1) {Gefuellter Knoten};
    %Kombinierte Formatierung
    \node[draw, fill=blue!10, thick] at (10,-1) {Formatierter Knoten};
\end{tikzpicture}
```

Ausgabe

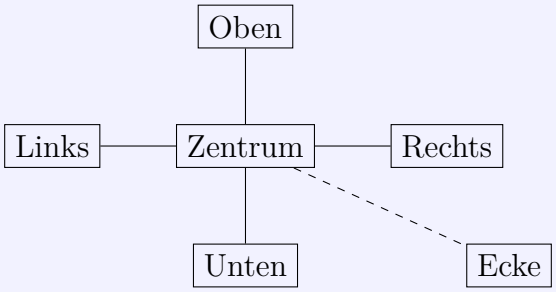


5.2 Positionierung von Knoten

Knoten können relativ zueinander positioniert werden, was besonders nützlich für Diagramme ist.

```
\begin{tikzpicture}
    % Zentraler Knoten
    \node[draw] (zentrum) at (2,2) {Zentrum};
    % Relative Positionierung (erfordert positioning-Bibliothek)
    \node[draw] (rechts) [right=of zentrum] {Rechts};
    \node[draw] (links) [left=of zentrum] {Links};
    \node[draw] (oben) [above=of zentrum] {Oben};
    \node[draw] (unten) [below=of zentrum] {Unten};
    % Verbindungen zwischen Knoten
    \draw (zentrum) -- (rechts);
    \draw (zentrum) -- (links);
    \draw (zentrum) -- (oben);
    \draw (zentrum) -- (unten);
    % Positionierung mit festem Abstand
    \node[draw] (ecke) [below right=1cm and 2cm of zentrum] {Ecke};
    \draw[dashed] (zentrum) -- (ecke);
\end{tikzpicture}
```

Ausgabe



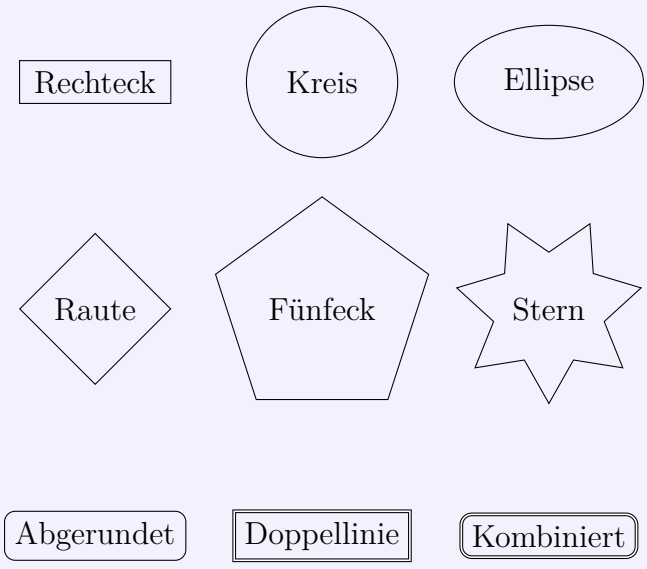
5.3 Verschiedene Knotenformen

Mit der `shapes`-Bibliothek stehen verschiedene Formen für Knoten zur Verfügung.

```
\begin{tikzpicture}[node distance=2.5cm]
    % Verschiedene Knotenformen (erfordert shapes-Bibliothek)
    \node[draw, rectangle, minimum width=2cm] at (0,0) {Rechteck};
    \node[draw, circle, minimum size=2cm] at (3,0) {Kreis};
    \node[draw, ellipse, minimum width=2.5cm, minimum height=1.5cm] at (6,0) {Ellipse};
    \node[draw, diamond, minimum size=2cm] at (0,-3) {Raute};
    \node[draw, regular polygon, regular polygon sides=5, minimum size=2cm] at (3,-3) {
        Fuenfeck};
    \node[draw, star, star points=7, star point ratio=0.6, minimum size=1.5cm] at (6,-3)
        {Stern};

    % Abgerundete Ecken und doppelte Linien
    \node[draw, rectangle, rounded corners, minimum width=2cm] at (0,-6) {Abgerundet};
    \node[draw, rectangle, double, minimum width=2cm] at (3,-6) {Doppellinie};
    \node[draw, rectangle, double, rounded corners, minimum width=2cm] at (6,-6) {
        Kombiniert};
\end{tikzpicture}
```

Ausgabe



5.4 Knoten auf Pfaden platzieren

Knoten können auch direkt auf Pfade gesetzt werden, was für Beschriftungen nützlich ist.

```
\begin{tikzpicture}
    % Pfad mit Beschriftungen
    \draw (0,0) -- node[above] {oben} node[below] {unten} (3,0);

    % Pfad mit Beschriftung in der Mitte
    \draw (4,0) -- node[midway, above] {Mitte} (7,0);

    % Verschiedene Positionen
    \draw (8,0) -- node[pos=0.25, above] {25\%}
                   node[pos=0.75, above] {75\%} (11,0);

    % Mehrere verbundene Linien mit Beschriftungen
    \draw (12,0) -- node[midway, above] {A} (13,1)
                 -- node[rotate=-45,  midway, above] {B} (14,0)
                 -- node[rotate=45, midway, below] {C} (15,1);
\end{tikzpicture}
```

Ausgabe



6 Farben in Ti*k*Z

Ti*k*Z unterstützt eine Vielzahl von Farben, die mit Prozentangaben gemischt werden können.

```
\begin{tikzpicture}
    % Grundlegende Farben
    \draw[red, ultra thick] (0,0) -- (1,0);
    \draw[green, ultra thick] (0,-0.5) -- (1,-0.5);
    \draw[blue, ultra thick] (0,-1) -- (1,-1);
    % Farbmischungen mit Prozentangabe
    \draw[red!50, ultra thick] (2,0) -- (3,0);              % 50% Rot
    \draw[blue!30, ultra thick] (2,-0.5) -- (3,-0.5);         % 30% Blau
    \draw[green!70, ultra thick] (2,-1) -- (3,-1);            % 70% Gruen
    % Mischung von zwei Farben
    \draw[red!30!blue, ultra thick] (4,0) -- (5,0);         % 30% Rot, 70% Blau
    \draw[yellow!50!green, ultra thick] (4,-0.5) -- (5,-0.5); % 50% Gelb, 50% Gruen
    % Transparenz
    \fill[red, opacity=0.5] (6,-0.5) circle (0.5);
    \fill[blue, opacity=0.5] (6.4,-0.5) circle (0.5);
    % Farbverlauf
    \shade[left color=blue, right color=yellow] (8,0) rectangle (10,-1);
    \shade[top color=red, bottom color=green] (11,0) rectangle (13,-1);
\end{tikzpicture}
```

Ausgabe

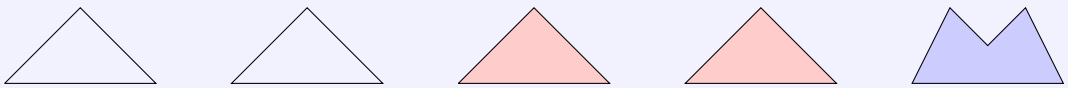


7 Verwendung der `cycle`-Funktion

Die `cycle`-Funktion schließt einen Pfad, indem sie eine Linie vom letzten zum ersten Punkt zieht. Dies ist besonders nützlich für geschlossene Formen und hat Vorteile gegenüber dem manuellen Zurückzeichnen zum Anfangspunkt.

```
\begin{tikzpicture}
    % Ohne cycle - manuelles Zurueckzeichnen zum Anfangspunkt
    \draw (0,0) -- (1,1) -- (2,0) -- (0,0);
    % Mit cycle - automatisches Schliessen des Pfades
    \draw (3,0) -- (4,1) -- (5,0) -- cycle;
    % Vorteile der cycle-Funktion:
    % 1. Bei Flaechenfuellung werden manuell geschlossene Pfade manchmal
    %    nicht als geschlossen erkannt
    \draw[fill=red!20] (6,0) -- (7,1) -- (8,0) -- (6,0);  % Kann problematisch sein
    % 2. Mit cycle ist die Fuellung garantiert korrekt
    \draw[fill=red!20] (9,0) -- (10,1) -- (11,0) -- cycle;
    % 3. Bei komplexen Pfaden ist cycle leichter zu warten
    \draw[fill=blue!20] (12,0) -- (12.5,1) -- (13,0.5) -- (13.5,1) -- (14,0) -- cycle;
\end{tikzpicture}
```

Ausgabe

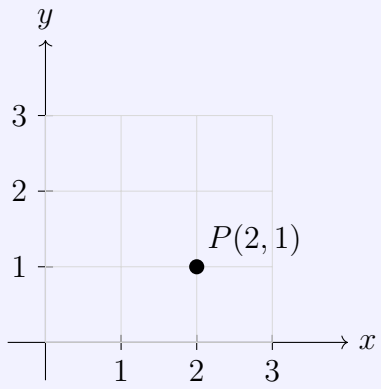


8 Praktische Beispiele

8.1 Koordinatensystem

```
\begin{tikzpicture}
    % Achsen zeichnen
    \draw[->] (-0.5,0) -- (4,0) node[right] {$x$};
    \draw[->] (0,-0.5) -- (0,4) node[above] {$y$};
    % Achsenbeschriftung
    \foreach \x in {1,2,3}{
        \draw (\x,0.1) -- (\x,-0.1) node[below] {$\x$};
    }
    \foreach \y in {1,2,3}{
        \draw (0.1,\y) -- (-0.1,\y) node[left] {$\y$};
    }
    % Gitter (optional)
    \draw[very thin, color=gray!30] (0,0) grid (3,3);
    % Beispielpunkt
    \fill (2,1) circle (0.1) node[above right] {$P(2,1)$};
\end{tikzpicture}
```

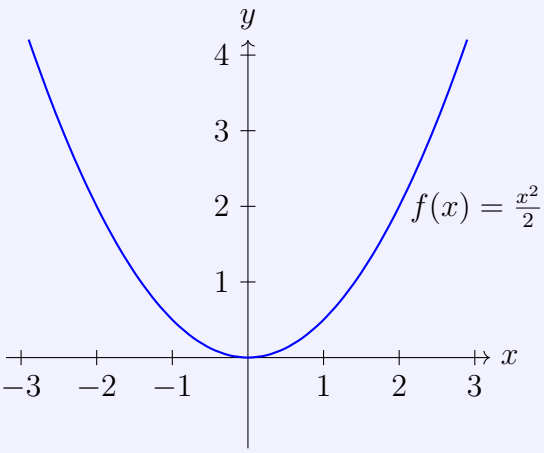
Ausgabe



8.2 Mathematisches Beispiel: Funktionsgraphen zeichnen

```
\begin{tikzpicture}
    %Koordinatensystem und Achsenbeschriftung
    \draw[->] (-3.2,0) -- (3.2,0) node[right] {$x$};
    \draw[->] (0,-1.2) -- (0,4.2) node[above] {$y$};
    \foreach \x in {-3,-2,-1,1,2,3}
        \draw (\x,0.1) -- (\x,-0.1) node[below] {$\x$};
    \foreach \y in {1,2,3,4}
        \draw (0.1,\y) -- (-0.1,\y) node[left] {$\y$};
    %Funktion zeichnen
    \draw[domain=-2.9:2.9, smooth, variable=\x, blue, thick] plot ({\x}, {\x*\x/2});
    %Beschriftung
    \node[right] at (2,2) {$f(x) = \frac{x^2}{2}$};
\end{tikzpicture}
```

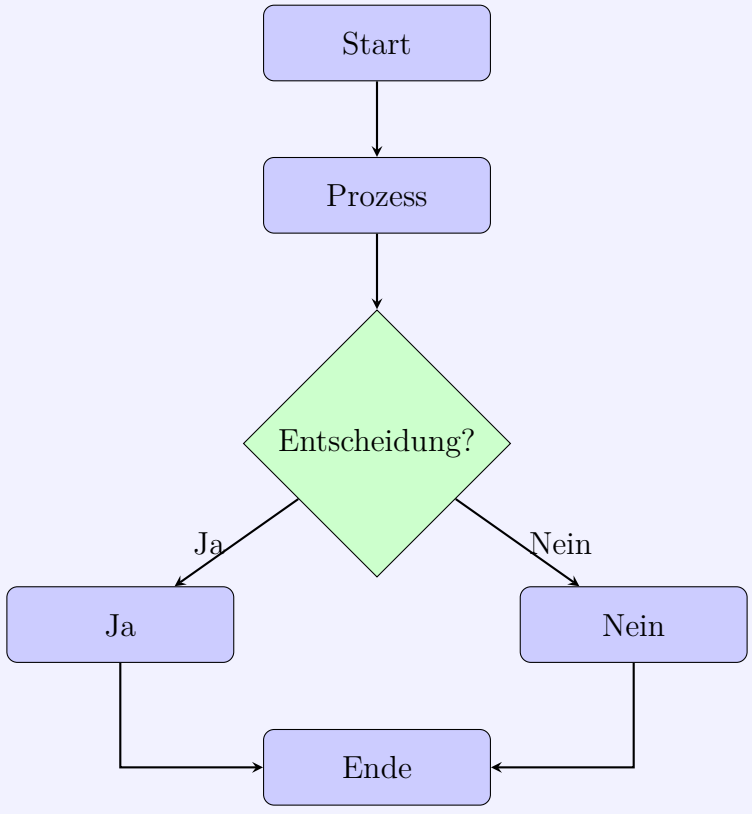
Ausgabe



8.3 Einfaches Flussdiagramm

```
\begin{tikzpicture}[
    % Globale Stileinstellungen
    block/.style={rectangle, draw, fill=blue!20, rounded corners,
                  minimum width=3cm, minimum height=1cm},
    decision/.style={diamond, draw, fill=green!20,
                     minimum width=3cm, minimum height=1cm},
    arrow/.style={thick,->,>=stealth}
]
    % Knoten definieren
    \node[block] (start) {Start};
    \node[block, below=of start] (process) {Prozess};
    \node[decision, below=of process] (decision) {Entscheidung?};
    \node[block, below left=of decision] (yes) {Ja};
    \node[block, below right=of decision] (no) {Nein};
    \node[block, below=2cm of decision] (end) {Ende};
    % Verbindungen
    \draw[arrow] (start) -- (process);
    \draw[arrow] (process) -- (decision);
    \draw[arrow] (decision) -- node[left] {Ja} (yes);
    \draw[arrow] (decision) -- node[right] {Nein} (no);
    \draw[arrow] (yes) |- (end);
    \draw[arrow] (no) |- (end);
\end{tikzpicture}
```

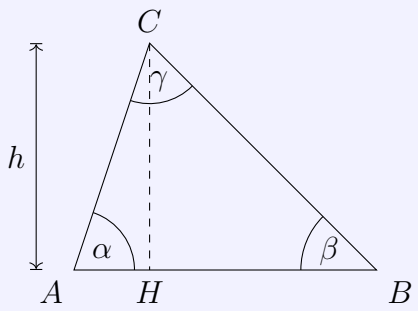
Ausgabe



8.4 Geometrisches Beispiel: Winkel

```
\begin{tikzpicture}
    % Dreieck zeichnen
    \coordinate (A) at (0,0);
    \coordinate (B) at (4,0);
    \coordinate (C) at (1,3);
    \draw (A) -- (B) -- (C) -- cycle;
    % Punkte beschriften
    \node[below left] at (A) {$A$};
    \node[below right] at (B) {$B$};
    \node[above] at (C) {$C$};
    % Winkel anzeigen
    \draw pic[draw, angle radius=0.8cm] {angle=B--A--C};
    \node at (0.375,0.25) {$\alpha$};
    \draw pic[draw, angle radius=1cm] {angle=C--B--A};
    \node at (3.375,0.25) {$\beta$};
    \draw pic[draw, angle radius=0.8cm] {angle=A--C--B};
    \node at (1.125,2.5) {$\gamma$};
    % Hoehe zeichnen
    \coordinate (H) at (1,0); % Hoehenfusspunkt
    \draw[dashed] (C) -- (H);
    \node[below] at (H) {$H$};
    % Hoehe beschriften
    \draw[|<->|] (-0.5,0) -- (-0.5,3) node[midway, left] {$h$};
\end{tikzpicture}
```

Ausgabe



9 Tipps für die Arbeit mit Ti*k*Z

Die Arbeit mit Ti*k*Z kann durch einige praktische Herangehensweisen deutlich erleichtert werden. Es empfiehlt sich, komplexe Grafiken schrittweise aufzubauen und regelmäßig zu testen, um Fehler frühzeitig zu erkennen und zu beheben. Für komplexe Strukturen bietet es sich an, benannte Knoten zu verwenden, da diese das spätere Referenzieren erheblich vereinfachen. Ausführliche Kommentare im Code helfen dabei, den Überblick zu behalten, besonders bei umfangreichen Grafiken.

Für wiederkehrende Elemente sollten eigene Stile definiert werden, um den Code übersichtlich zu halten und Änderungen global vornehmen zu können. Bei komplexeren Berechnungen von Positionen erweist sich die `calc`-Bibliothek als besonders nützlich. Sie ermöglicht arithmetische Operationen innerhalb von Koordinatenangaben. Für ein genaueres Platzie-

ren kann die `grid`-Option hilfreich sein, die durch den Befehl `\begin{tikzpicture}[help lines]` aktiviert wird und ein Hilfsgitter einblendet.

10 Zusammenfassung

Ti*k*Z ist ein mächtiges Werkzeug zur Erstellung von Grafiken in LaTeX, das durch seine Flexibilität und umfangreichen Funktionen überzeugt. Die wichtigsten Grundlagen, die in dieser Handreichung behandelt wurden, umfassen die `tikzpicture`-Umgebung als Rahmen für jegliche Grafikelemente sowie den `\draw`-Befehl zum Zeichnen von Linien und verschiedenen geometrischen Formen.
Für das Platzieren von Text und strukturierten Elementen dient der `\node`-Befehl, während die `cycle`-Funktion das korrekte Schließen von Pfaden gewährleistet. Die Füllung von Flächen erfolgt mittels `\fill` oder `\filldraw`, wobei verschiedene Farben und Linienstile zur individuellen Gestaltung beitragen können.
Mit diesen Grundlagen lassen sich bereits zahlreiche nützliche Grafiken für den MINT-Unterricht erstellen. Die in dieser Handreichung vorgestellten Beispiele können als Ausgangspunkt für eigene Grafiken dienen und flexibel an individuelle Bedürfnisse angepasst werden.